Field Hours

CSCE 711 Foundation of Modern Cryptography

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

One key, one ciphertext

Perfect secrecy is a definition, and the one-time pad is the mechanism that meets it for a single use. Let the message bits be 1011 and the key bits be 1101. Bitwise, 1 XOR 1 is 0, 0 XOR 1 is 1, 1 XOR 0 is 1, and 1 XOR 1 is 0, so the ciphertext is 0110. You decrypt by XOR with the same key: 0110 XOR 1101 returns 1011. Use the key once.

You use the count of keys to see why one ciphertext hides the message. A 4-bit message has 16 possible keys if the key is uniform and independent of the message. For any fixed message and any fixed ciphertext, exactly 1 of those 16 keys produces that ciphertext, namely the XOR of the two strings. So each ciphertext has probability 1/16 given the message. STAT 211 is the probability prerequisite, and CSCE 411 is the algorithms prerequisite.

Encrypt and decrypt by the same XOR.

Line up message and key -> XOR each bit once -> Send only the ciphertext -> XOR the key again

1011 XOR 1101

1. Message 1011 and key 1101. The four XOR bits are 0, 1, 1, and 0, so the ciphertext is 0110.

2. Decrypt: 0110 XOR 1101. The bits return 1, 0, 1, and 1, which is the message 1011.

3. There are 16 keys of length 4. Exactly 1 of them maps this message to this ciphertext. You do not reuse the key on a second message.

Problem 1. Message bits are 1011 and the key is 1101. What is the ciphertext, and what do you get if you XOR that ciphertext with 1101 again?

Answer. The ciphertext is 0110. XOR with 1101 again returns 1011. Each bit of 1011 XOR 1101 is 0, 1, 1, 0. The key is used once.

Problem 2. The message is 4 bits and the key is uniform. How many keys are there, and how many of them map one fixed message to one fixed ciphertext?

Answer. There are 16 keys. Exactly 1 of them works, so the ciphertext probability given the message is 1/16. STAT 211 and CSCE 411 are the prerequisites for this counting.

Lecture 2 · 90 minutes

Advantage is a subtraction

Indistinguishability is an experiment, and the mechanism is a hidden bit. You send two equal-length messages. The challenger encrypts one, chosen by a bit that is 0 or 1 with equal chance, and you guess that bit. Right on 6

of 10 trials means success rate 6/10, which is 3/5. Blind guessing is right with probability 1/2. Advantage is 3/5 minus 1/2, which equals 1/10.

You compare the advantage to a bound you wrote before the experiment. Suppose the bound is 1/20. Your advantage of 1/10 equals 2/20, which is larger than 1/20, so this adversary breaks the bound you set. An advantage of 0 would mean 5 of 10 expected from pure guessing, not a moral victory. You also refuse a comparison across unequal message lengths, because the experiment required equal length.

A bound you wrote before you counted wins.

This experiment

6 wins out of 10

Rate 3/5

Advantage 1/10

The bound

Bound was 1/20

1/10 equals 2/20

2/20 breaks 1/20

Six wins out of ten

1. You are right on 6 of 10 trials. 6/10 is 3/5. Blind guessing is 1/2.

2. Subtract: 3/5 minus 1/2 is 1/10. That is the advantage you report.

3. The bound on the page is 1/20. Since 1/10 is 2/20, the bound fails. You do not rerun the experiment with messages of different lengths.

Problem 1. An adversary is right on 6 of 10 trials. What is the success rate as a fraction in lowest terms, and what is the advantage over 1/2?

Answer. The success rate is 3/5. The advantage is 1/10, because 3/5 minus 1/2 is 1/10. Five wins out of 10 would have been advantage 0.

Problem 2. The written bound is 1/20 and the advantage is 1/10. Does the adversary break the bound? Express both with denominator 20.

Answer. Yes. 1/10 is 2/20, and 2/20 is larger than 1/20. The messages in the experiment also have to be equal length, or the comparison is refused.

Lecture 3 · 90 minutes

Sixteen strings out of 256

A pseudorandom generator stretches a short seed, and it does not give you the pad's perfect secrecy. The mechanism is expansion: a 4-bit seed has 16 possible values, and an 8-bit output has 256 possible strings. At most 16 of those 256 strings are generator outputs, a fraction of 16/256, which is 1/16. If you see a string outside

that set of 16, you know it was not produced by the generator.

You count fresh bits before you reuse a stretched string as a pad. One 8-bit generator output, from a 4-bit seed, is being asked to hide 8 message bits. The one-time pad would have required 8 fresh key bits, so the deficit is 8 minus 4, which is 4. What replaces the information-theoretic claim is a computational bound: a distinguisher you care about should have advantage under 1/16 on the test you stated.

Expansion is a count, not a new pad.

1. Count the seeds

2. Count the output strings

3. Divide to get the fraction

4. State a numeric bound

A four-bit seed, an eight-bit string

1. Seeds: 2 to the 4th is 16. Outputs: 2 to the 8th is 256. At most 16 outputs occur, and 16/256 is 1/16.

2. Using one such output to hide 8 message bits leaves a deficit of 4 fresh bits, because 8 minus 4 is 4.

3. You replace the perfect-secrecy claim with a bound, advantage under 1/16 for the distinguisher you named. A string outside the image set is not generator output.

Problem 1. A generator stretches a 4-bit seed to 8 bits. How many seeds are there, how many 8-bit strings are there, and what fraction of strings can be outputs?

Answer. 16 seeds and 256 strings. The fraction is 16/256, which is 1/16. A string outside those at most 16 outputs was not produced by the generator.

Problem 2. You hide 8 message bits with a pad derived from a 4-bit seed. How many fresh key bits is the construction short of a one-time pad?

Answer. 4. The pad would need 8 fresh bits, and 8 minus 4 is 4. The replacement claim is computational: advantage under a stated bound such as 1/16, not perfect secrecy.

Lecture 4 · 90 minutes

Two ciphertexts, the key cancels

Reusing a pad destroys the subtraction you just trusted. The mechanism of the failure is the XOR of two ciphertexts. Let the key be 0110, the first message 1010, and the second message 1100. Then 1010 XOR 0110 is 1100, and 1100 XOR 0110 is 1010. XOR the ciphertexts: 1100 XOR 1010 is 0110. XOR the messages: 1010 XOR 1100 is 0110. The two results match, so the key canceled.

A fresh key for the second message stops that cancellation from being about the messages alone. If the second key is 0011 instead of 0110, the second ciphertext is 1100 XOR 0011, which is 1111. Then the ciphertext XOR is 1100 XOR 1111, which is 0011, while the message XOR is still 0110. They differ, so you no longer read the message XOR directly. Security here means the hidden-bit advantage stays under a bound such as 1/20.

Reuse lets the messages show through.

Same key 0110

Ciphertext XOR is 0110

Message XOR is 0110

The key canceled

Second key 0011

Ciphertext XOR is 0011

Message XOR is still 0110

They no longer match

1010 and 1100 under one key

1. Key 0110. First ciphertext is 1010 XOR 0110, which is 1100. Second ciphertext is 1100 XOR 0110, which is 1010.

2. Ciphertext XOR is 1100 XOR 1010, which is 0110. Message XOR is 1010 XOR 1100, which is also 0110.

3. Change the second key to 0011 and the second ciphertext becomes 1111. Ciphertext XOR is then 0011, not the message XOR 0110.

Problem 1. Key 0110, messages 1010 and 1100. What is the XOR of the two ciphertexts, and what is the XOR of the two messages?

Answer. Both are 0110. 1010 XOR 0110 is 1100, 1100 XOR 0110 is 1010, and 1100 XOR 1010 is 0110. The matching message XOR means the key canceled.

Problem 2. The second key is 0011 instead, and the first ciphertext is still 1100. What is the second ciphertext, and what is the ciphertext XOR?

Answer. The second ciphertext is 1111, because 1100 XOR 0011 is 1111. The ciphertext XOR is 0011, because 1100 XOR 1111 is 0011. The message XOR remains 0110, so they differ. Keep the hidden-bit advantage under a bound such as 1/20.

Lecture 5 · 90 minutes

Exponents that invert

A public-key scheme uses a key you can publish and a different key you keep. The mechanism in this toy is exponentiation modulo n. Take primes 5 and 11, so n is 55. There are 40 integers below 55 coprime to 55. Private exponent 27 inverts public exponent 3 because 3 times 27 is 81, and 81 minus 2 times 40 is 1. Publishing 3 and 55 does not publish the factors 5 and 11.

You encrypt the message 2 by raising it to the public exponent. Compute 2 cubed, which is 8, and 8 mod 55 is 8, so the ciphertext is 8. Decryption raises 8 to the 27th power mod 55. You can build that power by squares: 8 squared is 64, and 64 minus 55 is 9. Then 9 squared is 81, and 81 minus 55 is 26, so 8 to the 4th is 26.

Check the inverse, then the ciphertext.

Multiply the primes -> Check 3 times 27 -> Cube the message -> Square up to exponent 27

Ciphertext 8 comes back to 2

1. n is 5 times 11, which is 55. 3 times 27 is 81, and 2 times 40 is 80, so 81 minus 80 is 1. The exponents invert.

2. Encrypt message 2 as 2 cubed mod 55, which is 8. Then 8 squared is 64, and 64 minus 55 is 9. 9 squared is 81, and 81 minus 55 is 26, so 8 to the 4th is 26.

3. Continue: 26 squared is 676, and 55 times 12 is 660, so 8 to the 8th is 16. 16 squared is 256, and 55 times 4 is 220, so 8 to the 16th is 36. Then 36 times 16 is 576, which is 26 mod 55; 26 times 9 is 234, which is 14 mod 55; 14 times 8 is 112, which is 2 mod 55.

Problem 1. Compute 3 times 27, compute 2 times 40, and subtract. What do you get, and why does it matter for exponents 3 and 27?

Answer. 81 minus 80 is 1. So 3 times 27 is 1 more than a multiple of 40, and 27 inverts 3 modulo 40. Publishing 3 and n=55 does not publish the factors 5 and 11.

Problem 2. The public exponent is 3, n is 55, and the message is 2. What is the ciphertext? After reducing 8 squared mod 55, what residue do you have?

Answer. The ciphertext is 8, because 2 cubed is 8 and 8 mod 55 is 8. 8 squared is 64, and 64 minus 55 is 9. Full decryption of 8 to the 27th mod 55 returns 2.

Lecture 6 · 90 minutes

Cube the signature

A signature is not encryption run backward as a privacy tool. The mechanism is a private exponent on a short digest, checked with the public exponent. Using the same toy, sign the digest 2: 2 to the 27th mod 55. Because 2 to the 20th is 1 mod 55, and 2 to the 7th is 128, which is 2 times 55 plus 18, the signature is 18. You check with public exponent 3.

Verification raises the signature to the public exponent. Compute 18 squared, which is 324. Then 324 minus 5 times 55: 5 times 55 is 275, and 324 minus 275 is 49. Now multiply by 18, because the public exponent is 3, so you need one more factor of 18. 49 times 18 is 882. Then 16 times 55 is 880, and 882 minus 880 is 2. The result 2 matches the digest, so you accept.

Verify in public, sign in private.

1. Raise digest 2 to 27

2. Reduce mod 55 to 18

3. Cube 18 mod 55

4. Accept only if you get 2

18 cubed returns 2

1. 2 to the 20th is 1 mod 55, and 2 to the 7th is 128, which is 110 plus 18. The signature on digest 2 is 18.

2. 18 squared is 324. Subtract 275, which is 5 times 55, and get 49. 49 times 18 is 882.

3. Subtract 880, which is 16 times 55, and get 2. The digest matches, so you accept. A result of anything but 2 is a reject.

Problem 1. Sign digest 2 with private exponent 27 modulo 55. 2 to the 20th is 1 mod 55, and 2 to the 7th is 128. What is the signature?

Answer. 18. 128 minus 2 times 55 is 128 minus 110, which is 18. Because 2 to the 20th contributes a factor of 1, 2 to the 27th is 18 mod 55.

Problem 2. Verify signature 18 by cubing it mod 55. What is 18 squared mod 55, and what is the final residue?

Answer. 18 squared is 324, and 324 minus 275 is 49, so the square is 49 mod 55. Then 49 times 18 is 882, and 882 minus 880 is 2. You accept because 2 matches the digest.

Lecture 7 · 90 minutes

Four rounds, one sixteenth

Zero knowledge convinces a verifier without handing over the secret. The mechanism is a challenge you could have simulated. In this toy the verifier sends 1 bit, 0 or 1. An honest prover who knows the secret answers either bit and is accepted. A cheating prover who can answer only 1 of the 2 bits succeeds with probability 1/2 on that round. Over 4 independent rounds the cheat probability is 1/2 to the 4th, which is 1/16.

The simulator is how you test the zero-knowledge claim. It picks the challenge bit first, then builds a transcript that accepts for that bit, with no help from the prover. On 1 round it can always do this for the bit it picked, which is 1 of 2 bits. That transcript is not evidence the simulator knows the secret. It is evidence the view is easy to fake.

What four challenge rounds change.

One round: cheat chance 1/2

Simulator picks the bit first

Honest prover passes either bit

Four rounds: cheat chance 1/16

A cheater faces four bits

1. Each round the verifier sends 1 bit. A cheater who prepared only 1 of the 2 answers succeeds with probability 1/2.

2. Four independent rounds multiply: 1/2 times 1/2 times 1/2 times 1/2 is 1/16.

3. The simulator picks its bit first and fakes that one answer. The fake transcript does not mean the simulator knows the secret. You still require all 4 rounds to pass.

Problem 1. A cheater succeeds on one challenge bit with probability 1/2. What is the success probability over 4 independent rounds?

Answer. 1/16. That is 1/2 to the 4th, or 1/2 times 1/2 times 1/2 times 1/2. You accept only when all 4 rounds pass.

Problem 2. The simulator picks 1 of 2 challenge bits before it builds the transcript. Does that transcript show the simulator knows the secret?

Answer. No. It shows the verifier's view can be faked for the bit the simulator already chose. Soundness is the separate 1/16 bound on a cheater across 4 rounds.

Lecture 8 · 90 minutes

What you can now compute

You add secret inputs using additive shares. That is the mechanism. They want 6 plus 7, which is 13. Split 6 into 2 plus 4, and split 7 into 1 plus 6. A holds 2 and 1, and adds them to get a local sum of 3. B holds 4 and 6, and adds them to get 10. They add the local sums, 3 plus 10, and get 13.

Boneh's Cryptography I and the Boneh-Shoup book are the path you stay on. If definitions are unfriendly in week 1, use Christof Paar for that week only, then return. Your closing check is the pad: 1011 XOR 1101 equals 0110, and the share sum 3 plus 10 equals 13. You also recompute advantage 1/10 from 6 of 10 trials before you call the definition done. If either closing number is wrong on your page, this course is not finished.

Shares that reconstruct to 13.

Split 6 into 2 and 4 -> Split 7 into 1 and 6 -> Add locally to 3 and 10 -> Add those to get 13

Local sums 3 and 10

1. x is 6, shared as 2 and 4. y is 7, shared as 1 and 6. Neither party holds both pieces of either input.

2. A adds 2 plus 1 and gets 3. B adds 4 plus 6 and gets 10. 3 plus 10 is 13, and 6 plus 7 is 13.

3. You close by also checking 1011 XOR 1101, which is 0110, and 6 wins out of 10, which is advantage 1/10. Boneh is the path after a week of Paar if you need it.

Problem 1. Party A holds shares 2 and 1. Party B holds shares 4 and 6. What are the two local sums, and what is the reconstructed total? Check it against 6 plus 7.

Answer. The local sums are 3 and 10. The total is 13, because 3 plus 10 is 13, and 6 plus 7 is also 13. A did not see 7, and B did not see 6.

Problem 2. The closing pad check uses message 1011 and key 1101. What is the ciphertext? Separately, 6 wins out of 10 trials gives what advantage over 1/2?

Answer. The ciphertext is 0110. The advantage is 1/10, because 6/10 is 3/5, and 3/5 minus 1/2 is 1/10. If either number is wrong, this course is not finished.